



What Are Some Good Debugging Techniques in Python?

Debugging is an essential skill for any Python developer. Understanding foundational techniques can help you identify and fix issues efficiently.

Debugging is the process of identifying and resolving errors or bugs in your code. It involves examining the code, understanding the flow, and using tools to pinpoint the source of problems.

In this lesson, we'll go over common debugging techniques you can use in your next Python project.

Using the print function and f-strings

First, using the `print()` function and f-strings at various points in your code can help you understand the flow and state of variables. For example:

```
def add(a, b):  
    result = a + b  
    print(f'Adding {a} and {b} gives {result}')
```

By printing the values of `a`, `b`, and `result`, you can verify that the function behaves as expected.

Interactive Debugging with the `pdb` Module

Next, you can utilize Python's built-in `pdb` module for interactive debugging:

```
import pdb  
  
def divide(a, b):  
    pdb.set_trace()  
    return a / b  
  
print(divide(10, 2))
```

By setting a trace with the `.set_trace()` method, you can step through the code, inspect variables, and understand the program's behavior.

If you run the code above, you'll see some output showing the location of the file you're running, the line where you called the `.set_trace()` method and the code immediately after it, and an interactive `pdb` prompt:

```
> /Users/fcc/Desktop/debugging.py(5)divide()  
-> return a / b  
(Pdb)
```

If you enter `help` into the prompt, you'll see a list of commands you can use:

```
(Pdb) help
```



Python Certification					Understanding Error Handling		
a	cl	debug	help	ll	quit	s	unt
alias	clear	disable	ignore	longlist	r	source	until
args	commands	display	interact	n	restart	step	up
b	condition	down	j	next	return	tbreak	w
break	cont	enable	jump	p	retval	u	whatis
bt	continue	exit	l	pp	run	unalias	where

Miscellaneous **help** topics:

```
=====
exec  pdb
```

Then you can use the commands to debug your code.

For example, if you want to look at the type of elements throughout your code at that moment, you can use the `whatis` command:

```
(Pdb) whatis a
<class 'int'>
(Pdb) whatis divide
Function divide
```

As you can see, by the time you run `.set_trace()`, the type of the parameter `a` is an integer, and `divide` is a function.

Then to continue execution of your code, you can use the `continue` command, or one of its aliases, `cont` or `c`:

```
(Pdb) continue
5.0
```

IDE Debugging Tools

Many Integrated Development Environments (IDEs) offer advanced debugging tools, such as breakpoints, step execution, and variable inspection.

Using VS Code Debugger

If you use VS Code, you can set breakpoints in your code and run the debugger to pause execution at those points. Here's how to debug the same `divide` function:

Step 1: Set up your code Create a file called `main.py` with the following content:

```
def divide(a, b):
    result = a / b
    return result

print(divide(10, 2))
print(divide(15, 3))
```

Step 2: Set a breakpoint

1. Click in the gutter (left margin) next to line 2 (`result = a / b`) to set a breakpoint
2. A red dot will appear, indicating the breakpoint is set

Step 3: Start debugging



Step 4: Inspect variables

- Hover over variables to see their current values
- Use the Variables panel on the left to see all local variables
- Use the Debug Console at the bottom to evaluate expressions

Step 5: Step through code

Use the debug toolbar to:

- **Continue (F5)**: Resume execution until the next breakpoint
- **Step Over (F10)**: Execute the current line and move to the next
- **Step Into (F11)**: Enter into function calls
- **Step Out (Shift+F11)**: Exit the current function

IDE debugging tools provide a visual interface to examine the state of your program, making it easier to identify and fix issues compared to using print statements alone.

By mastering these foundational debugging techniques - using `print()` statements, the `pdb` module, and IDE tools - you can effectively identify and resolve issues in your Python code. Each technique has its place: `print()` statements for quick checks, `pdb` for interactive exploration, and IDE debuggers for visual inspection.

Questions

What is the purpose of using the `print()` function in debugging?

To execute code faster

To display variable values and program flow

To compile the code

To terminate the program

What does the `pdb` module in Python provide?

A graphical user interface

A way to compile Python code

An interactive debugging environment

A method to encrypt code

What advantage do IDE debugging tools have over other methods like `print()` statements?



Python Certification



Understanding Error Handling

They prevent syntax errors.

They fix bugs in your code automatically.

They provide a visual interface to inspect the state of your program.

Check your answer

Ask for Help